



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria
e Scienza dell'Informazione

Fundamentals of Robotics

ROS programming – Part 3

Luigi Palopoli & Nicu Sebe



Time to write our own code



Creation of messages and services

- Let us try to see how to create our own services and messages
- Specifically
 - **msg:** msg files are simple text files that describe the fields of a ROS message. They can be used to generate messages in different languages (stored in the msg directory of a package)
 - **srv:** describe a service and are composed of a request and a response (stored in the srv directory of a package)
- Messages are text files with a fixed type and a fixed name per line
 - int8, int16, int32, int64 (plus uint*)
 - float32, float64
 - string
 - time, duration
 - other msg files
 - variable-length array[] and fixed-length array[C]
 - in addition there is a Header type that contains a timestamp and a coordinate frame information (commonly used in ROS)



Example of messages and services

- An example of a message using header and string primitives is the following

```
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

- A service is very similar, except that it contains request and response.

```
int64 A
int64 B
---
int64 Sum
```



Creation of a message

- Let us create a message in our beginner_tutorials package.

```
> roscd beginner_tutorials  
> mkdir msg  
> echo "int64 num" > msg/Num.msg
```

- Our message will have to be translated into C++, Python, etc,
- First edit your package.xml uncommenting a couple of lines (the first used at build time and the second used at runtime)

```
<build_depend>message_generation</build_depend>  
<exec_depend>message_runtime</exec_depend>
```

- Now, modify CMakeLists.txt to add a dependence from message generation

```
# Do not just add this to your CMakeLists.txt, modify the existing text to add message_generation before the closing parenthesis  
find_package(catkin REQUIRED COMPONENTS  
  roscpp  
  rospy  
  std_msgs  
  message_generation  
)
```



Creation of a message

Let us make sure to export the message run-time dependency

```
catkin_package(  
  ...  
  CATKIN_DEPENDS message_runtime ...  
  ...)
```

We can now uncomment the line of code to add message files

```
# add_message_files(  
#   FILES  
#   Message1.msg  
#   Message2.msg  
# )
```

```
add_message_files(  
  FILES  
  Num.msg  
)
```

One last line to force the message generation

```
# generate_messages(  
#   DEPENDENCIES  
#   std_msgs  
# )
```

```
generate_messages(  
  DEPENDENCIES  
  std_msgs  
)
```



Creation of a message

- We can verify if the message has been created by

```
> rosmmsg show beginner_tutorials/Num
```

- If you do not remember the package you can as well use

```
> rosmmsg show Num
```



Creating a service

The creation of a service goes through similar steps

```
> roscd beginner_tutorials  
> mkdir srv
```

- We can copy a service from another package

```
> roscp rospy_tutorials AddTwoInts.srv srv/AddTwoInts.srv
```

- We follow the same steps as we did for message generation to create dependencies (the steps are common)

```
<build_depend>message_generation</build_depend>  
<exec_depend>message_runtime</exec_depend>
```

```
# Do not just add this to your CMakeLists.txt, modify the existing text to add message_generation  
before the closing parenthesis  
find_package(catkin REQUIRED COMPONENTS  
  roscpp  
  rospy  
  std_msgs  
  message_generation  
)
```




Creating a service

And then force the creation of the service

```
# add_service_files(  
#   FILES  
#   Service1.srv  
#   Service2.srv  
# )
```

```
add_service_files(  
  FILES  
  AddTwoInts.srv  
)
```

- We can check that the service has been created by:

```
> rossrv show beginner_tutorials/AddTwoInts
```



and now an application



A simple publisher subscriber

- Let us switch to the src directory and let us write our first application

```
> roscd beginner_tutorials  
> mkdir -p src
```

- We now activate our preferred editor and we start writing our application (talker.cpp and listener.cpp)



Talker.cpp

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>
int main(int argc, char **argv)
{
    ros::init(argc, argv, "talker");
    ros::NodeHandle n;

    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000),
    ros::Rate loop_rate(10);

    int count = 0;
    while (ros::ok())
    {
        std_msgs::String msg;

        std::stringstream ss;
        ss << "hello world " << count;
        msg.data = ss.str();

        ROS_INFO("%s", msg.data.c_str());

        chatter_pub.publish(msg);

        ros::spinOnce();

        loop_rate.sleep();
        ++count;
    }

    return 0;
}
```

Initialise ROS (we can do remapping through command line)

Initialise the node with the master (clean up upon destruction)

Declare to publish on a topic and get a publisher in return

How long will be my rate (in conjunction with sleep)

Creation of a message

Print on the screen

Publishing

Used for callbacks (not really needed here)



listener.cpp

```
#include "ros/ros.h"
#include "std_msgs/String.h"

void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
    ROS_INFO("I heard: [%s]", msg->data.c_str());
}

int main(int argc, char **argv)
{
    ros::init(argc, argv, "listener");

    ros::NodeHandle n;

    ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);

    ros::spin();

    return 0;
}
```

Callback code

Initialise ROS

Register

Subscribe

Keep waiting for callbacks



To build your package

- Just append the following to your CMakeLists.txt

```
add_executable(talker src/talker.cpp)
target_link_libraries(talker ${catkin_LIBRARIES})
add_dependencies(talker beginner_tutorials_generate_messages_cpp)

add_executable(listener src/listener.cpp)
target_link_libraries(listener ${catkin_LIBRARIES})
add_dependencies(listener beginner_tutorials_generate_messages_cpp)
```

- This will create the two executable. Now we need to create a dependency between the executable and the message generation

```
add_dependencies(talker beginner_tutorials_generate_messages_cpp)
```

- You may have dependencies also from other packages. You can sort them out by

```
target_link_libraries(talker ${catkin_LIBRARIES})
```



Build

- We can now build by

```
# In your catkin workspace  
> cd ~/catkin_ws  
> catkin_make  
> source devel/setup.bash
```

- To run the newly created node just type

```
roslaunch beginner_tutorials talker
```

```
roslaunch beginner_tutorials talker
```